Computer Architecture
CS 622 / EE 520
Spring 2026

Lecture 3

Shahid Masud

Topics

- Performance of Computer Systems
- Energy and Power Computations
- CPI and MIPS Rating
- Amdahl's Law Calculations
- SPEC Benchmarks
- ----
- RISC-V ISA Description

Chap 1 of textbook

Simple Computer Performance Measures

- Time to execute a program – in seconds
- Clock Speed of Computer – Mega Hertz or Giga Hz
- Comparing Benchmarks
- In terms of the width of Data Bus (16 bit, 32 bit, 64 bit)
- In terms of the size of different memory (8GB / 256GB)
  - Cache size, Levels of Cache
  - RAM, DRAM, SRAM
  - Hard disk, Online, Offline, Cloud
  - SSD, etc.
- In terms of multiple cores and multiple I/O available
- Software
  - Algorithm
  - Language / Compiler
  - Optimization – Parallel, Vector, etc.

In terms of Resources

What is MFLOPS or Giga Flops or Tera Flops?

FLOPS = Floating Point Linear Operations Per Second

$$\text{MFLOPS}\,rate = \frac{Number\ of\ executed\ floating\ point\ operations\ in\ a\ program}{Execution\ time\ \times 10^6}$$

Design Principles of Modern Computers

- Take Advantage of Parallelism
  - e.g. multiple processors, disks, memory banks, pipelining, multiple functional execution units
- Principle of Locality
  - Reuse of data and instructions (i.e. in Caches)
- Focus on the Common Case
  - Make commonly used functions fast; accelerators and Amdahl's Law

$$Speedup_{overall} = \frac{1}{(1 - Fraction_{enhanced}) + \frac{Fraction_{enhanced}}{Speedup_{enhanced}}}$$

$$Speedup = \frac{1}{(1 - f) + \frac{f}{SU_F}}$$

Energy and Power Calculations and Comparison

CPU Architecture Today

Heat becoming un unmanageable problem

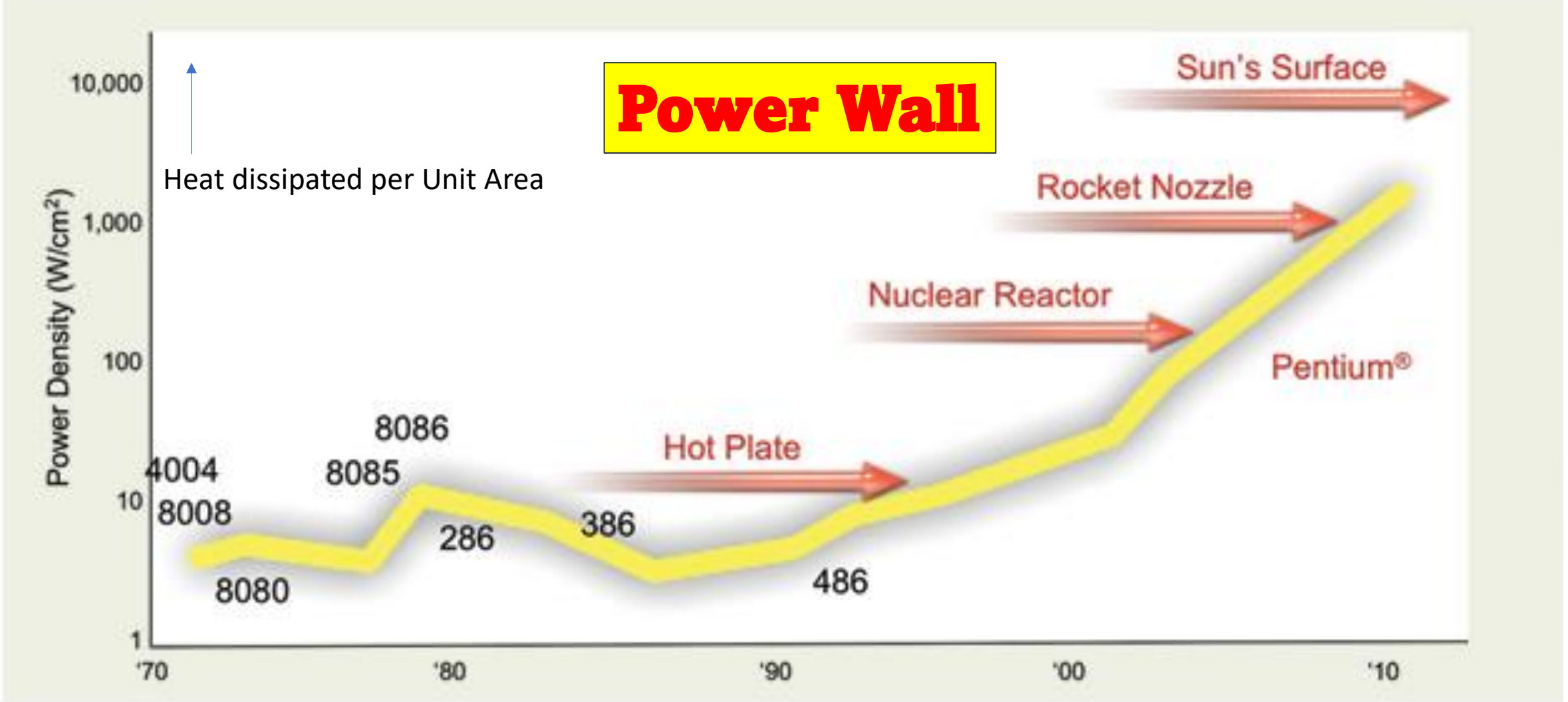


Figure 1. In CPU architecture today, heat is becoming an unmanageable problem. (Courtesy of Pat Gelsinger, Intel Developer Forum, Spring 2004)

Energy and Power in a Microprocessor

Energy and Power Within a Microprocessor

For CMOS chips, the traditional primary energy consumption has been in switching transistors, also called *dynamic energy*. The energy required per transistor is proportional to the product of the capacitive load driven by the transistor and the square of the voltage:

$$\text{Energy}_{\text{dynamic}} \propto \text{Capacitive load} \times \text{Voltage}^2$$

This equation is the energy of pulse of the logic transition of $0 \rightarrow 1 \rightarrow 0$ or $1 \rightarrow 0 \rightarrow 1$. The energy of a single transition ($0 \rightarrow 1$ or $1 \rightarrow 0$) is then:

$$\text{Energy}_{\text{dynamic}} \propto 1/2 \times \text{Capacitive load} \times \text{Voltage}^2$$

(no frequency here)

The power required per transistor is just the product of the energy of a transition multiplied by the frequency of transitions:

$$\text{Power}_{\text{dynamic}} \propto 1/2 \times \text{Capacitive load} \times \text{Voltage}^2 \times \text{Frequency switched}$$

For a fixed task, slowing clock rate reduces power, but not energy.

Clearly, dynamic power and energy are greatly reduced by lowering the voltage, so voltages have dropped from 5 V to just under 1 V in 20 years. The capacitive load is a function of the number of transistors connected to an output and the technology, which determines the capacitance of the wires and the transistors.

Inside Book cover

Computer Architecture Formulas

1. *CPU time* = Instruction count × Clock cycles per instruction × Clock cycle time
2. X is n times faster than Y: n = Execution time$_Y$ / Execution time$_X$ = Performance$_X$ / Performance$_Y$
3. *Amdahl's Law:* $$\text{Speedup}_{\text{overall}} = \frac{\text{Execution time}_{\text{old}}}{\text{Execution time}_{\text{new}}} = \frac{1}{(1 - \text{Fraction}_{\text{enhanced}}) + \frac{\text{Fraction}_{\text{enhanced}}}{\text{Speedup}_{\text{enhanced}}}}$$
4. $\text{Energy}_{\text{dynamic}} \propto 1/2 \times \text{Capacitive load} \times \text{Voltage}^2$
5. $\text{Power}_{\text{dynamic}} \propto 1/2 \times \text{Capacitive load} \times \text{Voltage}^2 \times \text{Frequency switched}$
6. $\text{Power}_{\text{static}} \propto \text{Current}_{\text{static}} \times \text{Voltage}$
7. *Availability* = Mean time to fail / (Mean time to fail + Mean time to repair)
8. *Die yield* = Wafer yield $\times 1 / (1 + \text{Defects per unit area} \times \text{Die area})^N$

   where Wafer yield accounts for wafers that are so bad they need not be tested and N is a parameter called the process-complexity factor, a measure of manufacturing difficulty. N ranges from 11.5 to 15.5 in 2011.
9. *Means—arithmetic (AM), weighted arithmetic (WAM), and geometric (GM):*

$$\text{AM} = \frac{1}{n} \sum_{i=1}^{n} \text{Time}_i \quad \text{WAM} = \sum_{i=1}^{n} \text{Weight}_i \times \text{Time}_i \quad \text{GM} = \sqrt[n]{\prod_{i=1}^{n} \text{Time}_i}$$

   where Time_i is the execution time for the ith program of a total of n in the workload, Weight_i is the weighting of the ith program in the workload.
10. *Average memory-access time* = Hit time + Miss rate × Miss penalty
11. *Misses per instruction* = Miss rate × Memory access per instruction
12. *Cache index size:* 2^{index} = Cache size / (Block size × Set associativity)
13. *Power Utilization Effectiveness (PUE)* of a Warehouse Scale Computer = $\frac{\text{Total Facility Power}}{\text{IT Equipment Power}}$

Later

Example of Energy calculation

Example Some microprocessors today are designed to have adjustable voltage, so a 15% reduction in voltage may result in a 15% reduction in frequency. What would be the impact on dynamic energy and on dynamic power?

Answer Because the capacitance is unchanged, the answer for energy is the ratio of the voltages

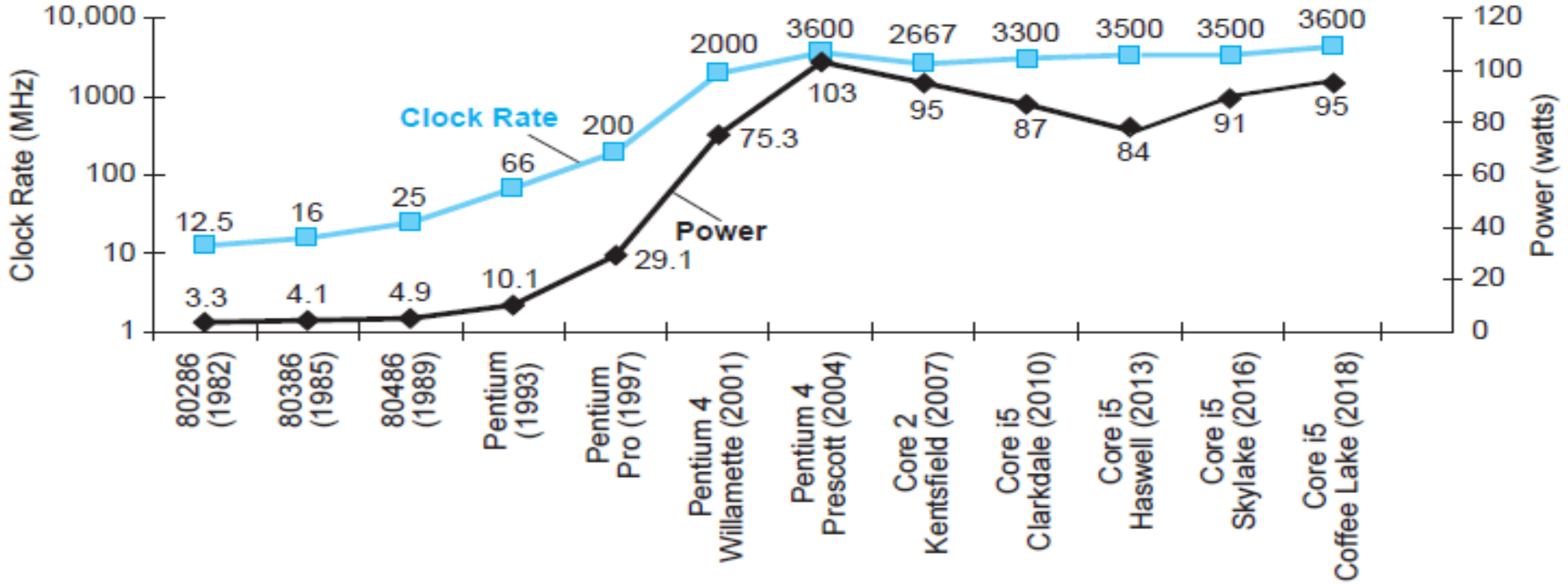
$$\frac{\text{Energy}_{\text{new}}}{\text{Energy}_{\text{old}}} = \frac{(\text{Voltage} \times 0.85)^2}{\text{Voltage}^2} = 0.85^2 = 0.72$$

which reduces energy to about 72% of the original. For power, we add the ratio of the frequencies

$$\frac{\text{Power}_{\text{new}}}{\text{Power}_{\text{old}}} = 0.72 \times \frac{(\text{Frequency switched} \times 0.85)}{\text{Frequency switched}} = 0.61$$

shrinking power to about 61% of the original.

CPU Power Trends



- In CMOS IC technology

$$\text{Power} = \text{Capacitive load} \times \text{Voltage}^2 \times \text{Frequency}$$

×30

5V → 1V

×1000

Reducing Power

- Suppose a new CPU has
  - 85% of capacitive load of old CPU
  - 15% voltage and 15% frequency reduction

$$\frac{P_{new}}{P_{old}} = \frac{C_{old} \times 0.85 \times (V_{old} \times 0.85)^2 \times F_{old} \times 0.85}{C_{old} \times V_{old}^{\ 2} \times F_{old}} = 0.85^4 = 0.52$$

- The power wall
  - We can't reduce voltage further
  - We can't remove more heat
- How else can we improve performance?

Dealing with Power Dissipation in Microprocessors

Clock control of Inactive modules

Dynamic Voltage and Frequency Sacling

Low Power Modes in:
DRAM
Clock Frequency
Disk Speed
CPU sleep mode

1. *Do nothing well.* Most microprocessors today turn off the clock of inactive modules to save energy and dynamic power. For example, if no floating-point instructions are executing, the clock of the floating-point unit is disabled. If some cores are idle, their clocks are stopped.
2. *Dynamic voltage-frequency scaling (DVFS).* The second technique comes directly from the preceding formulas. PMDs, laptops, and even servers have periods of low activity where there is no need to operate at the highest clock frequency and voltages. Modern microprocessors typically offer a few clock frequencies and voltages in which to operate that use lower power and energy. Figure 1.12 plots the potential power savings via DVFS for a server as the workload shrinks for three different clock rates: 2.4, 1.8, and 1 GHz. The overall server power savings is about 10%–15% for each of the two steps.
3. *Design for the typical case.* Given that PMDs and laptops are often idle, memory and storage offer low power modes to save energy. For example, DRAMs have a series of increasingly lower power modes to extend battery life in PMDs and laptops, and there have been proposals for disks that have a mode that spins more slowly when unused to save power. However, you cannot access DRAMs or disks in these modes, so you must return to fully active mode to read or write, no matter how low the access rate. As mentioned, microprocessors for PCs have been designed instead for heavy use at high operating temperatures, relying on on-chip temperature sensors to detect when activity should be reduced automatically to avoid overheating. This "emergency slowdown" allows manufacturers to design for a more typical case and then rely on this safety mechanism if someone really does run programs that consume much more power than is typical.

Eg. Energy Saving due to DVFS

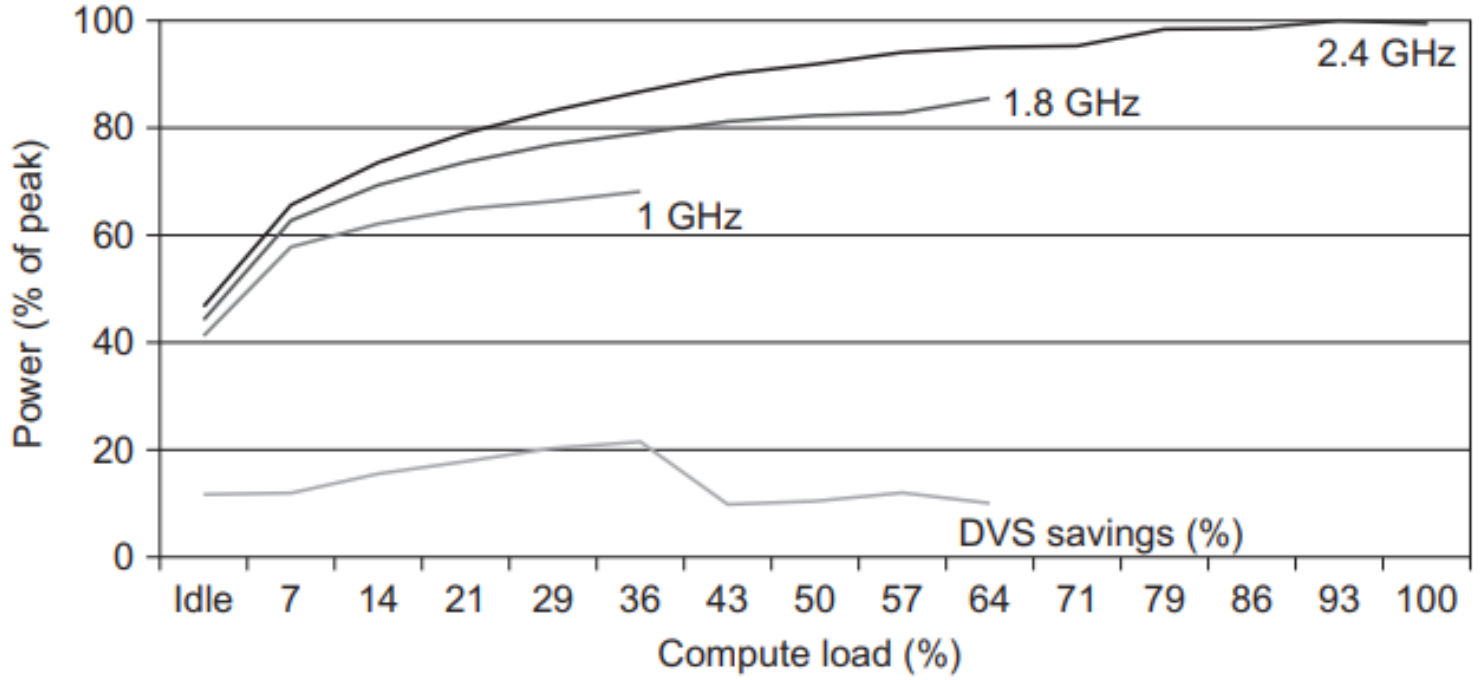


Figure 1.12 Energy savings for a server using an AMD Opteron microprocessor, 8 GB of DRAM, and one ATA disk. At 1.8 GHz, the server can handle at most up to two-thirds of the workload without causing service-level violations, and at 1 GHz, it can safely handle only one-third of the workload (Figure 5.11 in Barroso and Hölzle, 2009).

Dark Silicon in Computer Architecture

The Shift in Computer Architecture Because of Limits of Energy

As transistor improvement decelerates, computer architects must look elsewhere for improved energy efficiency. Indeed, given the energy budget, it is easy today to design a microprocessor with so many transistors that they cannot all be turned on at the same time. This phenomenon has been called *dark silicon*, in that much of a chip cannot be unused ("dark") at any moment in time because of thermal constraints. This observation has led architects to reexamine the fundamentals of processors' design in the search for a greater energy-cost performance.

In the electronics industry, dark silicon is the amount of circuitry of an integrated circuit that cannot be powered-on at the nominal operating voltage for a given thermal design power (TDP) constraint. (Wikipedia)

Reliability and Availability

Reliability and Availability

Module reliability is a measure of the continuous service accomplishment (or, equivalently, of the time to failure) from a reference initial instant. Therefore the *mean time to failure* (MTTF) is a reliability measure. The reciprocal of MTTF is a rate of failures, generally reported as failures per billion hours of operation, or *FIT* (for *failures in time*). Thus an MTTF of 1,000,000 hours equals $10^9/10^6$ or 1000 FIT. Service interruption is measured as *mean time to repair* (MTTR). *Mean time between failures* (MTBF) is simply the sum of MTTF+MTTR. Although MTBF is widely used, MTTF is often the more appropriate term. If a collection of modules has exponentially distributed lifetimes—meaning that the age of a module is not important in probability of failure—the overall failure rate of the collection is the sum of the failure rates of the modules.

Module availability is a measure of the service accomplishment with respect to the alternation between the two states of accomplishment and interruption. For nonredundant systems with repair, module availability is

$$\text{Module availability} = \frac{\text{MTTF}}{(\text{MTTF} + \text{MTTR})}$$

Dependability

- Module reliability
  - Mean time to failure (MTTF)
  - Mean time to repair (MTTR)
  - Mean time between failures (MTBF) = MTTF + MTTR
  - Availability = MTTF / MTBF

E.g. – Reliability of a Storage System

Example Assume a disk subsystem with the following components and MTTF:

- 10 disks, each rated at 1,000,000-hour MTTF
- 1 ATA controller, 500,000-hour MTTF
- 1 power supply, 200,000-hour MTTF
- 1 fan, 200,000-hour MTTF
- 1 ATA cable, 1,000,000-hour MTTF

Using the simplifying assumptions that the lifetimes are exponentially distributed and that failures are independent, compute the MTTF of the system as a whole.

Answer The sum of the failure rates is

$$\text{Failure rate}_{\text{system}} = 10 \times \frac{1}{1{,}000{,}000} + \frac{1}{500{,}000} + \frac{1}{200{,}000} + \frac{1}{200{,}000} + \frac{1}{1{,}000{,}000}$$

$$= \frac{10+2+5+5+1}{1{,}000{,}000 \text{ hours}} = \frac{23}{1{,}000{,}000} = \frac{23{,}000}{1{,}000{,}000{,}000 \text{ hours}}$$

FIT = Failure in Time

or 23,000 FIT. The MTTF for the system is just the inverse of the failure rate

$$\text{MTTF}_{\text{system}} = \frac{1}{\text{Failure rate}_{\text{system}}} = \frac{1{,}000{,}000{,}000 \text{ hours}}{23{,}000} = 43{,}500 \text{ hours}$$

or just under 5 years.

CPI and MIPS as Performance Measures

Performance in terms of CPU Execution Time

- Elapsed time
  - Total response time, including all aspects
    - Processing, I/O, OS overhead, idle time
  - Determines system performance
- **CPU time**
  - Time spent processing a given job
    - Discounts I/O time, other jobs' shares
  - Comprises user CPU time and system CPU time
  - Different programs are affected differently by CPU and system performance

$$\textbf{CPU Execution Time} = \frac{\textbf{Instructions}}{\textbf{Program}} \times \frac{\textbf{Clock Cycles}}{\textbf{Instruction}} \times \frac{\textbf{Seconds}}{\textbf{Clock Cycle}}$$

- **Performance can be improved by:**
  - Reducing the number of clock cycles
  - Increasing clock rate
  - Hardware designer must find trade off of clock rate against cycle count

CPU Time - Example

- **Computer A**: 2.0 GHz clock, 10s CPU time
- Designing **Computer B:**
  - Aim for **6s CPU time**
  - Causes **1.2 ×** clock cycles
- **How much fast the clock of Computer B should be?**

$$\text{Clock Rate}_B = \frac{\text{Clock Cycles}_B}{\text{CPU Time}_B} = \frac{1.2 \times \text{Clock Cycles}_A}{6\text{s}}$$

$$\text{Clock Cycles}_A = \text{CPU Time}_A \times \text{Clock Rate}_A$$

$$= 10\text{s} \times 2\text{GHz} = 20 \times 10^9$$

$$\text{Clock Rate}_B = \frac{1.2 \times 20 \times 10^9}{6\text{s}} = \frac{24 \times 10^9}{6\text{s}} = 4\text{GHz}$$

CPI = Cycles Per Instruction

- Each microprocessor specifies the number of Clock Cycles or Clock Ticks that specific type of instructions will consume
- CPI is an average number considering occurrence of different types of instructions in a program

Instruction Count and CPI

$$\textbf{Clock Cycles} = \textbf{Instruction Count} \times \textbf{Cycles per Instruction}$$

$$\textbf{CPU Time} = \textbf{Instruction Count} \times \textbf{CPI} \times \textbf{Clock Cycle Time}$$

$$= \frac{\textbf{Instruction Count} \times \textbf{CPI}}{\textbf{Clock Rate}}$$

- Instruction Count I_c for a program
  - Determined by program, ISA and compiler
- Average cycles per instruction
  - Determined by CPU hardware
  - Different instructions have different CPI
  - Average CPI affected by instruction mix

CPU Time and CPI Example

- **Computer A**: Cycle Time = 250ps, CPI = 2.0
- **Computer B**: Cycle Time = 500ps, CPI = 1.2
- Same ISA
- Which computer is faster, and by how much?

$$\text{CPU Time}_A = \text{Instruction Count} \times \text{CPI}_A \times \text{Cycle Time}_A$$
$$= I \times 2.0 \times 250\text{ps} = I \times 500\text{ps}$$

A is faster…

$$\text{CPU Time}_B = \text{Instruction Count} \times \text{CPI}_B \times \text{Cycle Time}_B$$
$$= I \times 1.2 \times 500\text{ps} = I \times 600\text{ps}$$

$$\frac{\text{CPU Time}_B}{\text{CPU Time}_A} = \frac{I \times 600\text{ps}}{I \times 500\text{ps}} = 1.2$$

…by this much

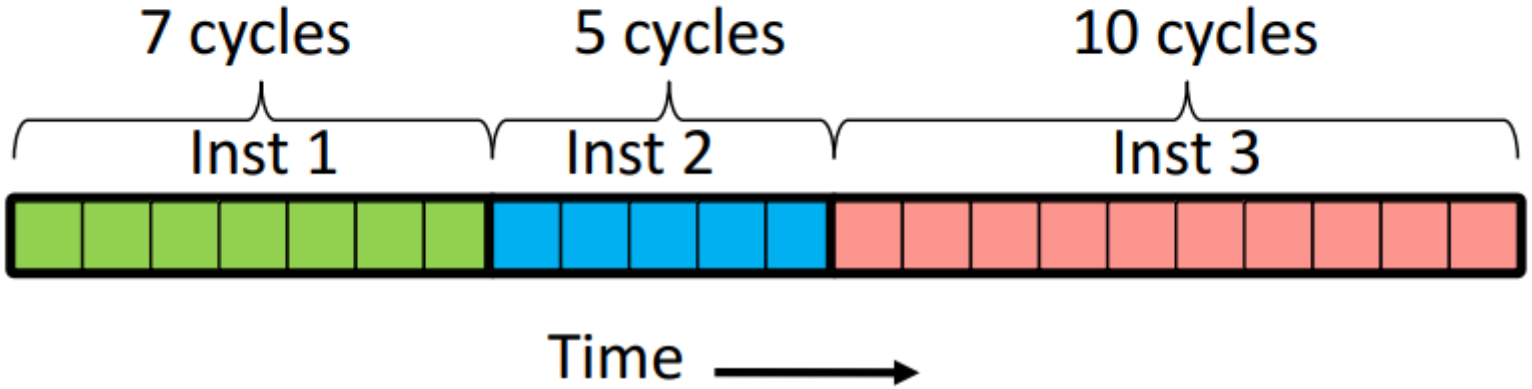
CPI - Averaging

- As different instruction classes take different numbers of cycles, we use **Average CPI**

$$\text{Clock Cycles} = \sum_{i=1}^{n} (\text{CPI}_i \times \text{Instruction Count}_i)$$

Average CPI Example

$$\text{Clock Cycles} = \sum_{i=1}^{n} (\text{CPI}_i \times \text{Instruction Count}_i)$$



Total clock cycles = 7+5+10 = 22
Total instructions = 3
CPI = 22/3 = 7.33

CPI is always an average over a large number of instructions

Formula for Overall CPI

The number of clock cycles varies for different types of instructions such as load, branch, mult etc.

Let CPI_i be the number of cycles required for instruction type i

Let I_i be the number of executed instructions of type i in a given program

The overall CPI is as follows:

$$CPI = \frac{\sum_{i=1}^{n}(CPI_i \times I_i)}{I_c}$$

Ic = Instruction Count

CPI Example

$$\textbf{Clock Cycles} = \sum_{i=1}^{n} (\textbf{CPI}_i \times \textbf{Instruction Count}_i)$$

- Alternative compiled code sequences using instructions in classes (or types) A, B, C

Class	A	B	C
CPI for class	1	2	3
I_C in sequence 1	2	1	2
I_C in sequence 2	4	1	1

- Sequence 1: $I_C = 5$
  - Clock Cycles
    = 2×1 + 1×2 + 2×3
    = 10
  - Avg. CPI = 10/5 = 2.0
- Sequence 2: $I_C = 6$
  - Clock Cycles
    = 4×1 + 1×2 + 1×3
    = 9
  - Avg. CPI = 9/6 = 1.5

CPI Question

Measurement	Computer A	Computer B
Instruction count	10 billion	8 billion
Clock rate	4 GHz	4 GHz
CPI	1.0	1.1

a. Which computer has the higher MIPS rating?

b. Which computer is faster?

$$MIPS\ rate = \frac{I_c}{T \times 10^6} = \frac{f}{CPI \times 10^6}$$

Factors Affecting CPI

The **processor time T** needed to execute a given program can be expressed as:

$$T = I_c \times CPI \times \tau$$

Time to transfer data depends on memory cycle time that can vary significantly viz a viz processor cycle time. Thus:

$$T = I_c \times [p + (m \times k)] \times \tau$$

Where:

p = number of processor cycles needed to decode and execute the instruction

m = number of memory references needed

k = ratio between memory cycle time and processor cycle time

$\tau = 1/f$; cycle time

I_c = instruction count

MIPS Formula and Example

- A common measure of CPU performance is the rate at which instructions are executed
- MIPS is abbreviation for Millions Instructions per second
- How many MIPS can a CPU do is its **MIPS rate**?
- In terms of equations:

$$MIPS\ rate = \frac{I_c}{T \times 10^6} = \frac{f}{CPI \times 10^6}$$

Example of MIPS and CPI

$$\text{MIPS rate} = \frac{I_c}{T \times 10^6} = \frac{f}{CPI \times 10^6} \qquad (2.3)$$

EXAMPLE 2.2 Consider the execution of a program that results in the execution of 2 million instructions on a 400-MHz processor. The program consists of four major types of instructions. The instruction mix and the *CPI* for each instruction type are given below, based on the result of a program trace experiment:

Instruction Type	*CPI*	Instruction Mix (%)
Arithmetic and logic	1	60
Load/store with cache hit	2	18
Branch	4	12
Memory reference with cache miss	8	10

The average *CPI* when the program is executed on a uniprocessor with the above trace results is $CPI = 0.6 + (2 \times 0.18) + (4 \times 0.12) + (8 \times 0.1) = 2.24$. The corresponding MIPS rate is $(400 \times 10^6)/(2.24 \times 10^6) \approx 178$.

Book Example, pg 54

There are two possibilities for improvement:

1. Improve all FP operations
2. Improve FPSQRT operation

Example Suppose we made the following measurements:

Frequency of FP operations = 25%

Average CPI of FP operations = 4.0

Average CPI of other instructions = 1.33

Frequency of FSQRT = 2%

CPI of FSQRT = 20

Assume that the two design alternatives are to decrease the CPI of FSQRT to 2 or to decrease the average CPI of all FP operations to 2.5. Compare these two design alternatives using the processor performance equation.

Answer First, observe that only the CPI changes; the clock rate and instruction count remain identical. We start by finding the original CPI with neither enhancement:

$$\text{CPI}_{\text{original}} = \sum_{i=1}^{n} \text{CPI}_i \times \left(\frac{\text{IC}_i}{\text{Instruction count}} \right)$$
$$= (4 \times 25\%) + (1.33 \times 75\%) = 2.0$$

We can compute the CPI for the enhanced FSQRT by subtracting the cycles saved from the original CPI:

$$\text{CPI}_{\text{with new FPSQR}} = \text{CPI}_{\text{original}} - 2\% \times (\text{CPI}_{\text{old FPSQR}} - \text{CPI}_{\text{of new FPSQR only}})$$
$$= 2.0 - 2\% \times (20 - 2) = 1.64$$

We can compute the CPI for the enhancement of all FP instructions the same way or by summing the FP and non-FP CPIs. Using the latter gives us

$$\text{CPI}_{\text{new FP}} = (75\% \times 1.33) + (25\% \times 2.5) = 1.625$$

Since the CPI of the overall FP enhancement is slightly lower, its performance will be marginally better. Specifically, the speedup for the overall FP enhancement is

$$\text{Speedup}_{\text{new FP}} = \frac{\text{CPU time}_{\text{original}}}{\text{CPU time}_{\text{new FP}}} = \frac{\text{IC} \times \text{Clock cycle} \times \text{CPI}_{\text{original}}}{\text{IC} \times \text{Clock cycle} \times \text{CPI}_{\text{new FP}}}$$
$$= \frac{\text{CPI}_{\text{original}}}{\text{CPI}_{\text{new FP}}} = \frac{2.00}{1.625} = 1.23$$

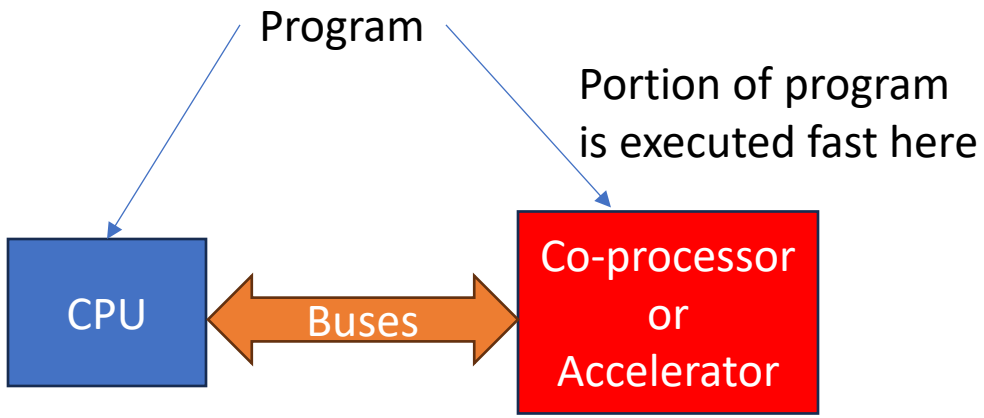
Amdahl’s Law for Performance

Accelerators, Multiprocessors and Multicores

- Performance Enhancement through:
  - Custom Hardware Accelerators
  - Instruction Set Extension
- Multicore Microprocessors
  - More than one processor per chip
- Requires explicitly parallel programming
  - Compare with instruction level parallelism
    - Hardware executes multiple instructions at once
    - Hidden from the programmer
  - Hard to do
    - Programming for performance
    - Load balancing
    - Optimizing communication and synchronization

Scenarios for Amdahl's Law

Scenario 1



Special / Complex operations:

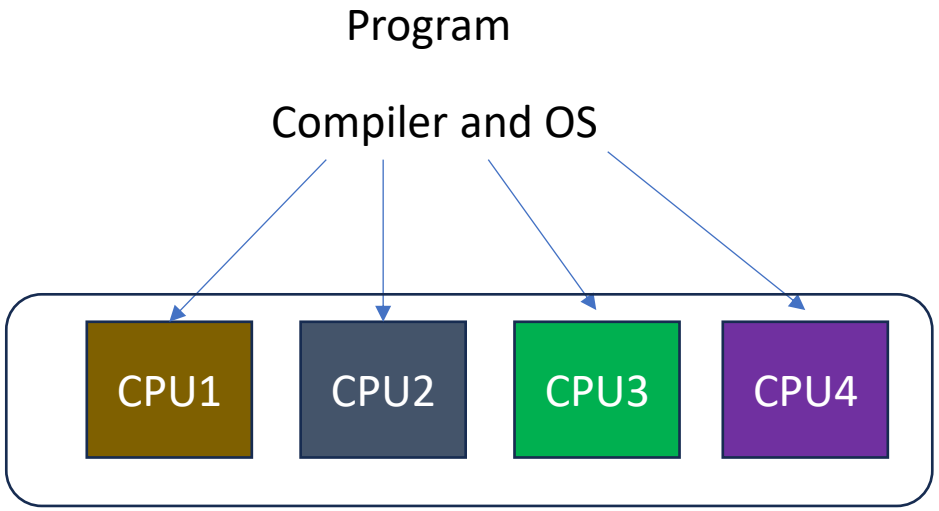
Floating point

Matrix multiplication

Convolution for AI

Hashing for Crypto

Scenario 2



Program is executed over multiple CPU in parallel

Amdahl's Law for Enhanced Performance

- Gives an idea of improvement in a program when more cores and additional hardware functionality is added.
- Separates the fraction of program code that can be parallelized / improved and the fraction that cannot be parallelized / improved.

$$Speedup = \frac{Performance\ after\ enhancement}{Performance\ before\ enhancement} = \frac{Execution\ time\ before\ enhancement}{Execution\ time\ after\ enhancement}$$

Formula for Amdahl's Law for Customized Speedup

- Suppose that a feature of the system is used during execution a fraction of the time f before enhancement
- The speedup of that feature after enhancement is SU_F
- Then overall speedup of the system is:

$$\boldsymbol{Speedup = \frac{1}{(1-f) + \frac{f}{SU_F}}}$$

Example Suppose that we want to enhance the processor used for web serving. The new processor is 10 times faster on computation in the web serving application than the old processor. Assuming that the original processor is busy with computation 40% of the time and is waiting for I/O 60% of the time, what is the overall speedup gained by incorporating the enhancement?

Answer $\text{Fraction}_{\text{enhanced}} = 0.4;\ \text{Speedup}_{\text{enhanced}} = 10;\ \text{Speedup}_{\text{overall}} = \dfrac{1}{0.6 + \dfrac{0.4}{10}} = \dfrac{1}{0.64} \approx 1.56$

Amdahl's Law for Multicore / Parallel Processing

- Let T be the total execution time of the program using a single processor.
- Then the speedup using a parallel processor with N processors that exploit the parallelizable portion of the program

- $$\textbf{Speedup} = \frac{\textbf{Time to execute program on a single processor}}{\textbf{Time to execute program on N parallel processors}}$$
- $$= \frac{T(1-f)+Tf}{T(1-f)+\frac{Tf}{N}}$$
- $$= \frac{1}{(1-f)+\frac{f}{N}}$$

Important Observation:

- When f is small, the use of parallel processors has little effect
- As N approaches infinity, speedup is bound by $1/(1-f)$, so that there are diminishing returns for more processors

Question using Amdahl's Law

A simple design problem illustrates it well. Suppose a program runs in 100 seconds on a computer, with multiply operations responsible for 80 seconds of this time. How much do I have to improve the speed of multiplication if I want my program to run five times faster?

Problem with Amdahl's Law

- Improving one aspect of a computer and expecting a proportional improvement in overall performance

$$T_{improved} = \frac{T_{affected}}{improvement\ factor} + T_{unaffected}$$

Practice Amdahl's Law Questions

Question 1:

Suppose that a computing task makes extensive use of Floating point (FP) computations with 40% execution time consumed in FP. With a new co-processor, the FP is sped up by a factor k. What is the maximum speed up possible with this FP co-processor.

Question 2:

A program takes 100 sec to run on a computer. 80 sec out of 100 sec are spent on multiplication operation. How much speed of multiplication has to improve to make program execute 5 times faster.

SPEC Benchmarks and CPU Performance

SPEC Benchmarks

- Benchmarks are used to evaluate performance of full computer systems. CPI measure is only for microprocessors.
- SPEC benchmarks are standard in Computer Architecture study.
- A benchmark contains several programs in a particular class of computing, i.e. integer calculations, floating-point computations, etc.
- SPEC Rating = Geometric Mean (N^{th} root) of product of performance obtained from running all programs in a particular class in the benchmark.

Why Benchmarks?

- Due to differences in instruction sets, the instruction execution rate is not a valid means of comparing performance of different architectures in the form of MIPS or MFLOPS.
- Performance of a program may not be useful in determining how that processor will perform on a very different type of application.
- Benchmarks provide guidance to customers to decide which system to buy and useful support to vendors and designers to design systems that meet benchmark goals
- Characteristics of benchmark programs:
  - Written in high-level languages hence portable across machines
  - Representative of a particular kind of programming domain or paradigm
  - Can be measured easily
  - It has a wide distribution

SPEC Website

https://www.spec.org/spec/

SPEC Products Results News Info Search... Sea

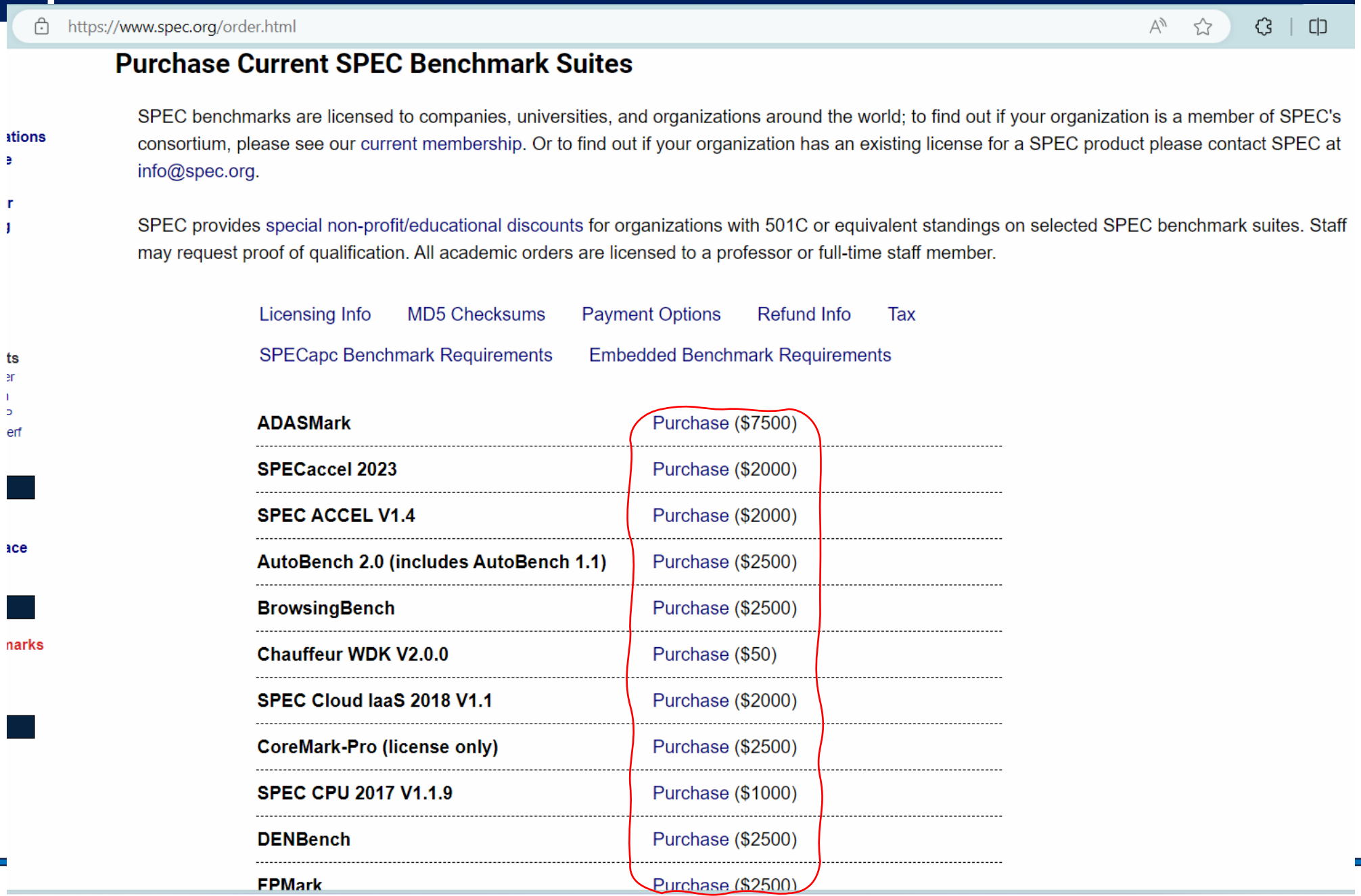
SPEC Membership About SPEC

The Standard Performance Evaluation Corporation (SPEC) was founded in 1988 by a handful of workstation vendors who recognized the desperate need for realistic, standardized performance tests.

SPEC has grown to become one of the world's leading computing performance standardization bodies, with a membership of more than 120 leading computer hardware and software vendors, educational institutions, research organizations, and government agencies worldwide. Each quarter, SPEC publishes hundreds of different performance results spanning a variety of system performance disciplines.

SPEC believes that the user community benefits greatly from an objective series of applications-oriented tests that serve as common reference points during system evaluation. By making these benchmarks relatively easy to use, SPEC encourages the widespread publication of performance results.

Cost of Spec benchmarks



https://www.spec.org/order.html

Purchase Current SPEC Benchmark Suites

SPEC benchmarks are licensed to companies, universities, and organizations around the world; to find out if your organization is a member of SPEC's consortium, please see our current membership. Or to find out if your organization has an existing license for a SPEC product please contact SPEC at info@spec.org.

SPEC provides special non-profit/educational discounts for organizations with 501C or equivalent standings on selected SPEC benchmark suites. Staff may request proof of qualification. All academic orders are licensed to a professor or full-time staff member.

Licensing Info | MD5 Checksums | Payment Options | Refund Info | Tax

SPECapc Benchmark Requirements | Embedded Benchmark Requirements

ADASMark	Purchase ($7500)
SPECaccel 2023	Purchase ($2000)
SPEC ACCEL V1.4	Purchase ($2000)
AutoBench 2.0 (includes AutoBench 1.1)	Purchase ($2500)
BrowsingBench	Purchase ($2500)
Chauffeur WDK V2.0.0	Purchase ($50)
SPEC Cloud IaaS 2018 V1.1	Purchase ($2000)
CoreMark-Pro (license only)	Purchase ($2500)
SPEC CPU 2017 V1.1.9	Purchase ($1000)
DENBench	Purchase ($2500)
FPMark	Purchase ($2500)

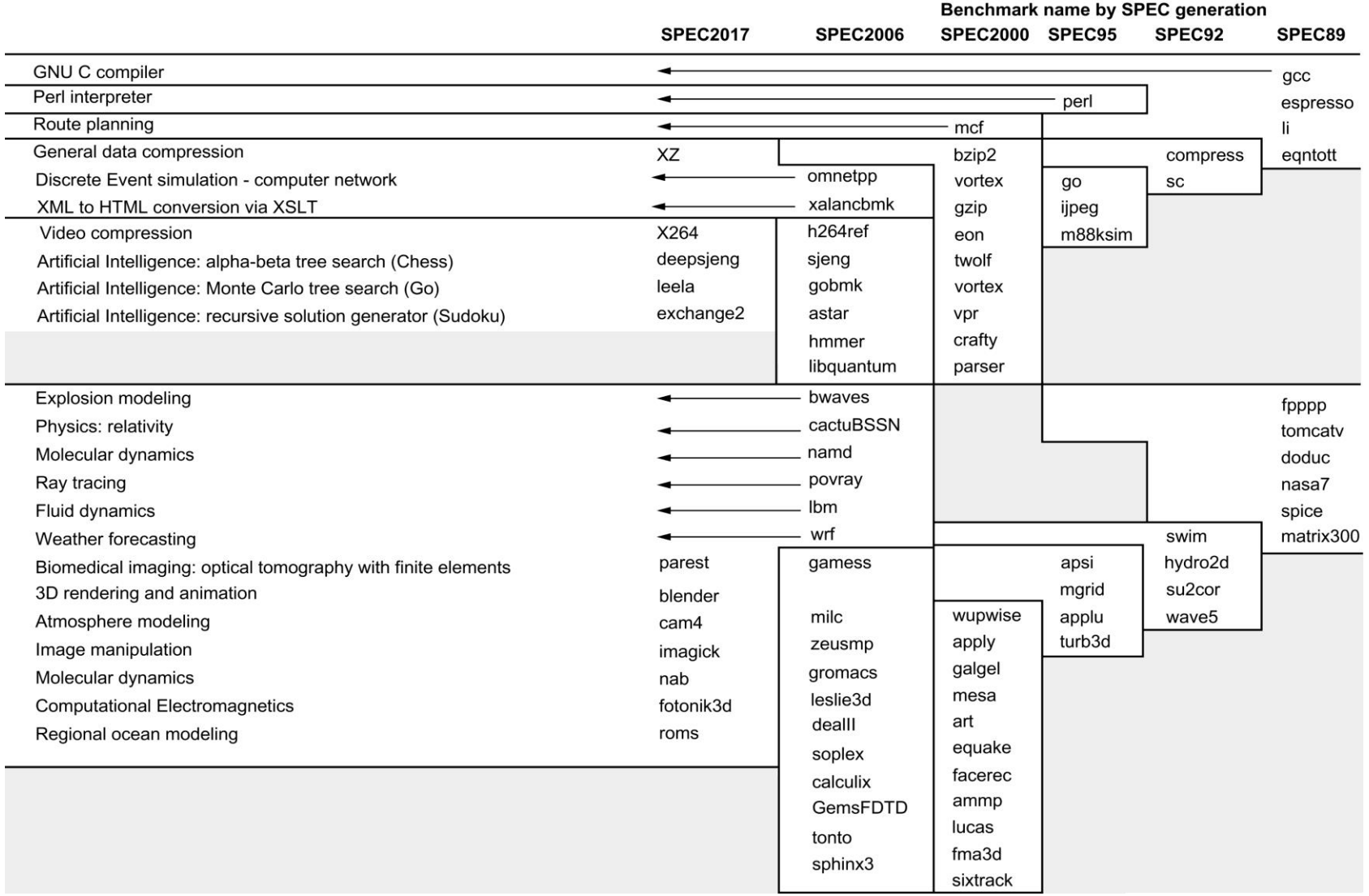


Figure 1.17 SPEC2017 programs and the evolution of the SPEC benchmarks over time, with integer programs above the line and floating-point programs below the line. Of the 10 SPEC2017 integer programs, 5 are written in C, 4 in C++., and 1 in Fortran. For the floating-point programs, the split is 3 in Fortran, 2 in C++, 2 in C, and 6 in mixed C, C++, and Fortran. The figure shows all 82 of the programs in the 1989, 1992, 1995, 2000, 2006, and 2017 releases. Gcc is the senior citizen of the group. Only 3 integer programs and 3 floating-point programs survived three or more generations. Although a few are carried over from generation to generation, the version of the program changes and either the input or the size of the benchmark is often expanded to increase its running time and to avoid perturbation in measurement or domination of the execution time by some factor other than CPU time. The benchmark descriptions on the left are for SPEC2017 only and do not apply to earlier versions. Programs in the same row from different generations of SPEC are generally not related; for example, fpppp is not a CFD code like bwaves.

Category	Name	Measures performance of
Cloud	Cloud_IaaS 2016	Cloud using NoSQL database transaction and K-Means clustering using map/reduce
CPU	CPU2017	Compute-intensive integer and floating-point workloads
Graphics and workstation performance	SPECviewperf® 12	3D graphics in systems running OpenGL and Direct X
	SPECwpc V2.0	Workstations running professional apps under the Windows OS
	SPECapcSM for 3ds Max 2015™	3D graphics running the proprietary Autodesk 3ds Max 2015 app
	SPECapcSM for Maya® 2012	3D graphics running the proprietary Autodesk 3ds Max 2012 app
	SPECapcSM for PTC Creo 3.0	3D graphics running the proprietary PTC Creo 3.0 app
	SPECapcSM for Siemens NX 9.0 and 10.0	3D graphics running the proprietary Siemens NX 9.0 or 10.0 app
	SPECapcSM for SolidWorks 2015	3D graphics of systems running the proprietary SolidWorks 2015 CAD/CAM app
High performance computing	ACCEL	Accelerator and host CPU running parallel applications using OpenCL and OpenACC
	MPI2007	MPI-parallel, floating-point, compute-intensive programs running on clusters and SMPs
	OMP2012	Parallel apps running OpenMP
Java client/server	SPECjbb2015	Java servers
Power	SPECpower_ssj2008	Power of volume server class computers running SPECjbb2015
Solution File Server (SFS)	SFS2014	File server throughput and response time
	SPECsfs2008	File servers utilizing the NFSv3 and CIFS protocols
Virtualization	SPECvirt_sc2013	Datacenter servers used in virtualized server consolidation

Figure 1.18 Active benchmarks from SPEC as of 2017.

Spec Ratio

Rather than pick weights, we could normalize execution times to a reference computer by dividing the time on the reference computer by the time on the computer being rated, yielding a ratio proportional to performance. SPEC uses this approach, calling the ratio the SPECRatio. It has a particularly useful property that matches the way we benchmark computer performance throughout this text—namely, comparing performance ratios. For example, suppose that the SPECRatio of computer A on a benchmark is 1.25 times as fast as computer B; then we know

$$1.25 = \frac{\text{SPECRatio}_A}{\text{SPECRatio}_B} = \frac{\dfrac{\text{Execution time}_{\text{reference}}}{\text{Execution time}_A}}{\dfrac{\text{Execution time}_{\text{reference}}}{\text{Execution time}_B}} = \frac{\text{Execution time}_B}{\text{Execution time}_A} = \frac{\text{Performance}_A}{\text{Performance}_B}$$

Notice that the execution times on the reference computer drop out and the choice of the reference computer is irrelevant when the comparisons are made as a ratio, which is the approach we consistently use. Figure 1.19 gives an example.

Because a SPECRatio is a ratio rather than an absolute execution time, the mean must be computed using the *geometric* mean. (Because SPECRatios have no units, comparing SPECRatios arithmetically is meaningless.) The formula is

$$\text{Geometric mean} = \sqrt[n]{\prod_{i=1}^{n} sample_i}$$

$$\text{Geometric mean} = \sqrt[n]{\prod_{i=1}^{n} sample_i}$$

In the case of SPEC, $sample_i$ is the SPECRatio for program i. Using the geometric mean ensures two important properties:

1. The geometric mean of the ratios is the same as the ratio of the geometric means.
2. The ratio of the geometric means is equal to the geometric mean of the performance ratios, which implies that the choice of the reference computer is irrelevant.

Example of Geometric Mean

Benchmarks	Sun Ultra Enterprise 2 time (seconds)	AMD A10-6800K time (seconds)	SPEC 2006Cint ratio	Intel Xeon E5-2690 time (seconds)	SPEC 2006Cint ratio	AMD/Intel times (seconds)	Intel/AMD SPEC ratios
perlbench	9770	401	24.36	261	37.43	1.54	1.54
bzip2	9650	505	19.11	422	22.87	1.20	1.20
gcc	8050	490	16.43	227	35.46	2.16	2.16
mcf	9120	249	36.63	153	59.61	1.63	1.63
gobmk	10,490	418	25.10	382	27.46	1.09	1.09
hmmer	9330	182	51.26	120	77.75	1.52	1.52
sjeng	12,100	517	23.40	383	31.59	1.35	1.35
libquantum	20,720	84	246.08	3	7295.77	29.65	29.65
h264ref	22,130	611	36.22	425	52.07	1.44	1.44
omnetpp	6250	313	19.97	153	40.85	2.05	2.05
astar	7020	303	23.17	209	33.59	1.45	1.45
xalancbmk	6900	215	32.09	98	70.41	2.19	2.19
Geometric mean			31.91		63.72	2.00	2.00

Figure 1.19 SPEC2006Cint execution times (in seconds) for the Sun Ultra 5—the reference computer of SPEC2006—and execution times and SPECRatios for the AMD A10 and Intel Xeon E5-2690. The final two columns show the ratios of execution times and SPEC ratios. This figure demonstrates the irrelevance of the reference computer in relative performance. The ratio of the execution times is identical to the ratio of the SPEC ratios, and the ratio of the geometric means (63.7231.91/20.86 = 2.00) is identical to the geometric mean of the ratios (2.00). Section 1.11 discusses libquantum, whose performance is orders of magnitude higher than the other SPEC benchmarks.

Example: SPECspeed 2017 Integer benchmarks on a 1.8 GHz Intel Xeon E5-2650L

Description	*Name*	*Instruction Count x 10^9*	*CPI*	*Clock cycle time (seconds x 10^–9)*	*Execution Time (seconds)*	*Reference Time (seconds)*	*SPECratio*
Perl interpreter	perlbench	2684	0.42	0.556	627	1774	2.83
GNU C compiler	gcc	2322	0.67	0.556	863	3976	4.61
Route planning	mcf	1786	1.22	0.556	1215	4721	3.89
Discrete Event simulation - computer network	omnetpp	1107	0.82	0.556	507	1630	3.21
XML to HTML conversion via XSLT	xalancbmk	1314	0.75	0.556	549	1417	2.58
Video compression	x264	4488	0.32	0.556	813	1763	2.17
Artificial Intelligence: alpha-beta tree search (Chess)	deepsjeng	2216	0.57	0.556	698	1432	2.05
Artificial Intelligence: Monte Carlo tree search (Go)	leela	2236	0.79	0.556	987	1703	1.73
Artificial Intelligence: recursive solution generator (Sudoku)	exchange2	6683	0.46	0.556	1718	2939	1.71
General data compression	xz	8533	1.32	0.556	6290	6182	0.98
Geometric mean							2.36

Spec Benchmark Question

(b) Sun Blade X6250

Benchmark	Execution time (secs)	Execution time (secs)	Execution time (secs)	Reference time (secs)	Ratio	Rate
400.perlbench	497	497	497	9770	19.66	78.63
401.bzip2	613	614	613	9650	15.74	62.97
403.gcc	529	529	529	8050	15.22	60.87
429.mcf	472	472	473	9120	19.32	77.29
445.gobmk	637	637	637	10,490	16.47	65.87
456.hmmer	446	446	446	9330	20.92	83.68
458.sjeng	631	632	630	12,100	19.18	76.70
462.libquantum	614	614	614	20,720	33.75	134.98
464.h264ref	830	830	830	22,130	26.66	106.65
471.omnetpp	619	620	619	6250	10.10	40.39
473.astar	580	580	580	7020	12.10	48.41
483.xalancbmk	422	422	422	6900	16.35	65.40

Question: Given output of running Spec Benchmarks; find the Spec Rating of this computer?

Hint: Use Geometric Mean to combine all individual rates in last column and come up with one rating

RISC-V ISA from Appendix A

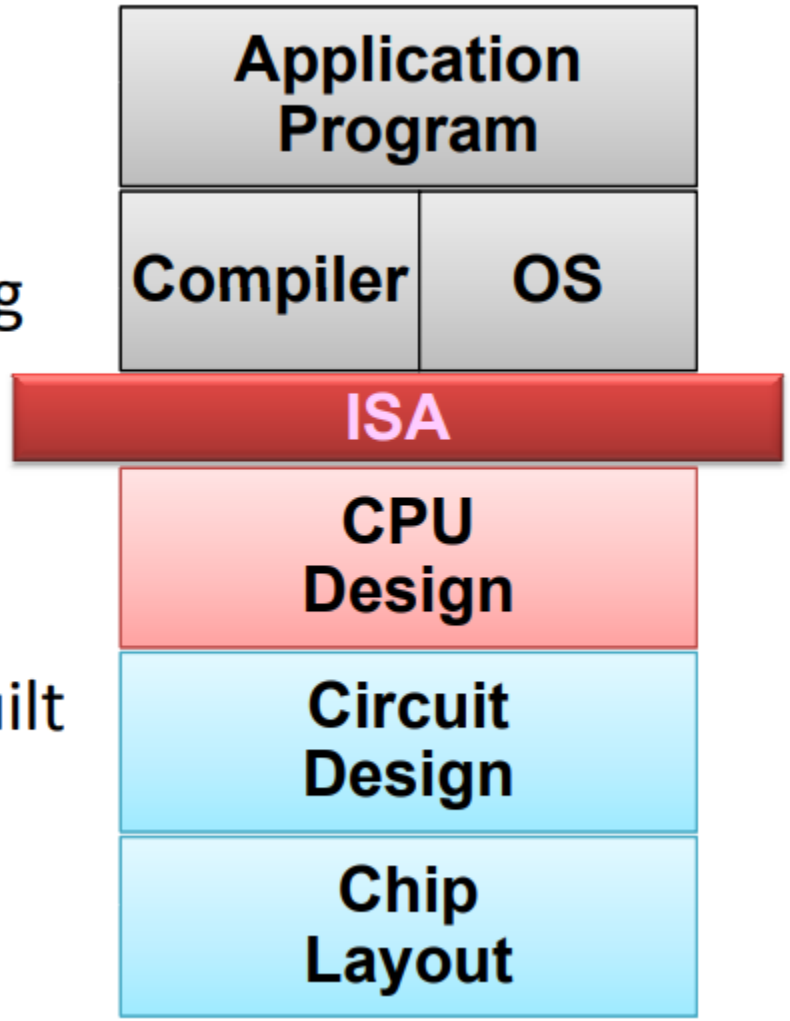
Instruction Set Architecture (ISA)

What is an Instruction Set Architecture?

- Lowest level of Computer Architecture that is visible to a programmer
- ISA comprises instructions in Assembly Language and Corresponding Binary Machine Code
- Primitive syntax compared to a high-level language
- It is easily interpreted and understood by the CPU
- Designed to maximize performance
- Represents features and performance of a CPU
- Designed to minimize cost and design time

Role of ISA within a Computer

- Assembly Language View
  - Processor state (RF, mem)
  - Instruction set and encoding
- Layer of Abstraction
  - Above: how to program machine - HLL, OS
  - Below: what needs to be built - tricks to make it run fast



Key:
RF = Register File
Mem = Memory
HLL = High Level Language
OS = Operating System

RISC-V ISA

Instruction Set Architecture: The Myopic View of Computer Architecture

We use the term *instruction set architecture* (ISA) to refer to the actual programmer-visible instruction set in this book. The ISA serves as the boundary between the software and hardware. This quick review of ISA will use examples from 80x86, ARMv8, and RISC-V to illustrate the seven dimensions of an ISA. The most popular RISC processors come from ARM (Advanced RISC Machine), which were in 14.8 billion chips shipped in 2015, or roughly 50 times as many chips that shipped with 80x86 processors. Appendices A and K give more details on the three ISAs.

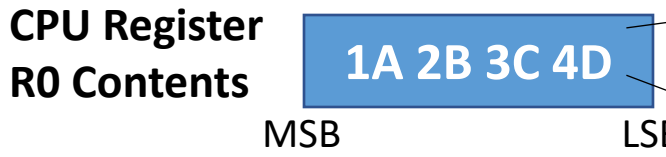
RISC-V ("RISC Five") is a modern RISC instruction set developed at the University of California, Berkeley, which was made free and openly adoptable in response to requests from industry. In addition to a full software stack (compilers, operating systems, and simulators), there are several RISC-V implementations freely available for use in custom chips or in field-programmable gate arrays. Developed 30 years after the first RISC instruction sets, RISC-V inherits its ancestors' good ideas—a large set of registers, easy-to-pipeline instructions, and a lean set of operations—while avoiding their omissions or mistakes. It is a free and open, elegant example of the RISC architectures mentioned earlier, which is why more than 60 companies have joined the RISC-V foundation, including AMD, Google, HP Enterprise, IBM, Microsoft, Nvidia, Qualcomm, Samsung, and Western Digital. We use the integer core ISA of RISC-V as the example ISA in this book.

RISC-V Instructions and Features

- Data Alignment
  - 32-bit bus vs 8-bit memory
  - Big Endian vs Little Endian
- Addressing Modes
- Type and Size of Operands
  - I, R types
- Operations available
  - Arithmetic
  - Logical
  - Shift
- Control Flow Instructions
  - Branch
  - Jump

Memory Access – Big Endian vs Little Endian

A CPU Register R0 is 32-bits wide.
It has data “1A2B3C4D” Hex.
Store the data in an 8-bit wide RAM that has 4 locations.
Store in Big-Endian style
or Store in Little-Endian Style



Location	Big Endian RAM
3	
2	
1	
0	

Location	Little Endian RAM
3	
2	
1	
0	

RISC-V Registers and Naming Conventions

Register	Name	Use	Saver
x0	zero	The constant value 0	N.A.
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	–
x4	tp	Thread pointer	–
x5-x7	t0-t2	Temporaries	Caller
x8	s0/fp	Saved register/frame pointer	Callee
x9	s1	Saved register	Callee
x10-x11	a0-a1	Function arguments/return values	Caller
x12-x17	a2-a7	Function arguments	Caller
x18-x27	s2-s11	Saved registers	Callee
x28-x31	t3-t6	Temporaries	Caller
f0-f7	ft0-ft7	FP temporaries	Caller
f8-f9	fs0-fs1	FP saved registers	Callee
f10-f11	fa0-fa1	FP function arguments/return values	Caller
f12-f17	fa2-fa7	FP function arguments	Caller
f18-f27	fs2-fs11	FP saved registers	Callee
f28-f31	ft8-ft11	FP temporaries	Caller

Figure 1.4 RISC-V registers, names, usage, and calling conventions. In addition to the 32 general-purpose registers (x0–x31), RISC-V has 32 floating-point registers (f0–f31) that can hold either a 32-bit single-precision number or a 64-bit double-precision number. The registers that are preserved across a procedure call are labeled "Callee" saved.

RISC-V is a Register-rich Architecture

RISC-V Instruction Formats and Types

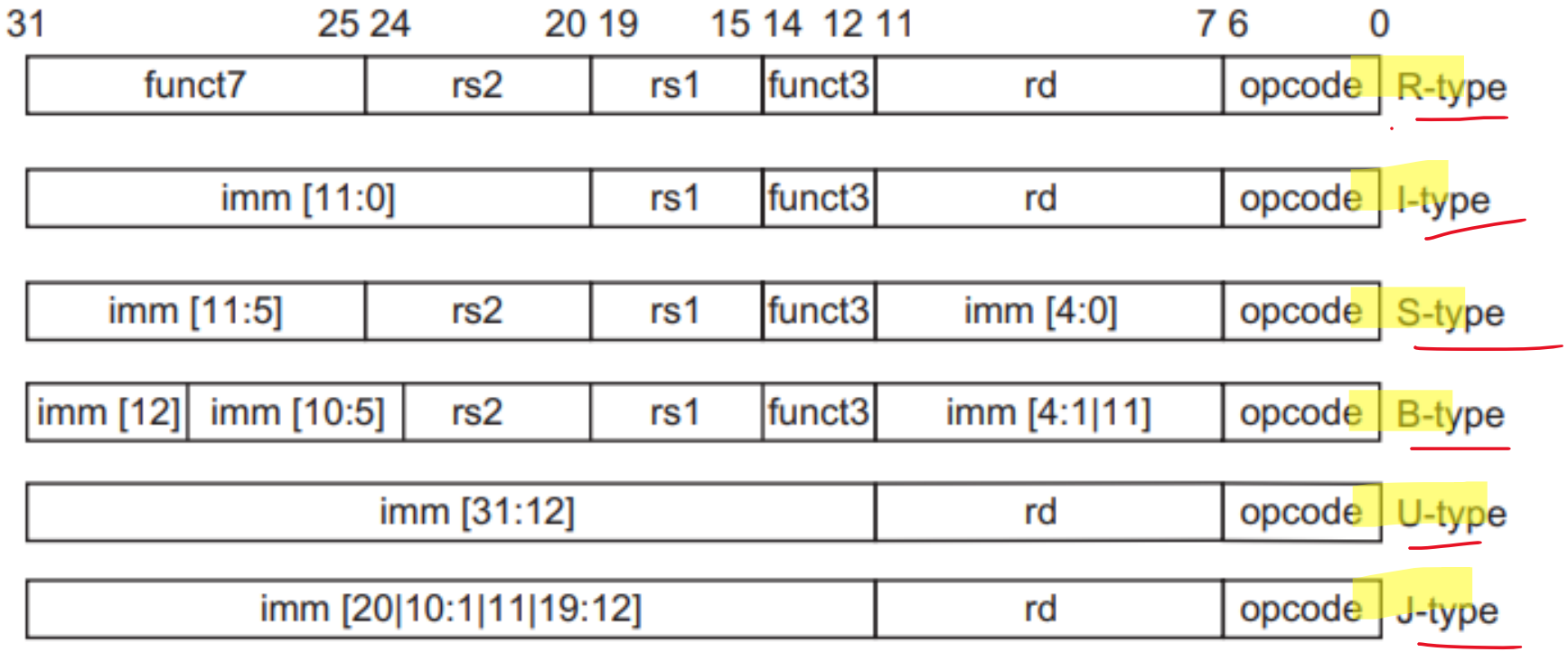


Figure 1.7 The base RISC-V instruction set architecture formats. All instructions are 32 bits long. The R format is for integer register-to-register operations, such as ADD, SUB, and so on. The I format is for loads and immediate operations, such as LD and ADDI. The B format is for branches and the J format is for jumps and link. The S format is for stores. Having a separate format for stores allows the three register specifiers (rd, rs1, rs2) to always be in the same location in all formats. The U format is for the wide immediate instructions (LUI, AUIPC).

RISC-V Data type Instructions

Instruction type/opcode	Instruction meaning
Data transfers	*Move data between registers and memory, or between the integer and FP or special registers; only memory address mode is 12-bit displacement + contents of a GPR*
`lb, lbu, sb`	Load byte, load byte unsigned, store byte (to/from integer registers)
`lh, lhu, sh`	Load half word, load half word unsigned, store half word (to/from integer registers)
`lw, lwu, sw`	Load word, load word unsigned, store word (to/from integer registers)
`ld, sd`	Load double word, store double word (to/from integer registers)
`flw, fld, fsw, fsd`	Load SP float, load DP float, store SP float, store DP float
`fmv._.x, fmv.x._`	Copy from/to integer register to/from floating-point register; "__" = S for single-precision, D for double-precision
`csrrw, csrrwi, csrrs, csrrsi, csrrc, csrrci`	Read counters and write status registers, which include counters: clock cycles, time, instructions retired

RISC-V Arithmetic / Logical Instructions

Arithmetic/logical	*Operations on integer or logical data in GPRs*
`add, addi, addw, addiw`	Add, add immediate (all immediates are 12 bits), add 32-bits only & sign-extend to 64 bits, add immediate 32-bits only
`sub, subw`	Subtract, subtract 32-bits only
`mul, mulw, mulh, mulhsu, mulhu`	Multiply, multiply 32-bits only, multiply upper half, multiply upper half signed-unsigned, multiply upper half unsigned
`div, divu, rem, remu`	Divide, divide unsigned, remainder, remainder unsigned
`divw, divuw, remw, remuw`	Divide and remainder: as previously, but divide only lower 32-bits, producing 32-bit sign-extended result
`and, andi`	And, and immediate
`or, ori, xor, xori`	Or, or immediate, exclusive or, exclusive or immediate
`lui`	Load upper immediate; loads bits 31-12 of register with immediate, then sign-extends
`auipc`	Adds immediate in bits 31–12 with zeros in lower bits to PC; used with `JALR` to transfer control to any 32-bit address
`sll, slli, srl, srli, sra, srai`	Shifts: shift left logical, right logical, right arithmetic; both variable and immediate forms
`sllw, slliw, srlw, srliw, sraw, sraiw`	Shifts: as previously, but shift lower 32-bits, producing 32-bit sign-extended result
`slt, slti, sltu, sltiu`	Set less than, set less than immediate, signed and unsigned

RISC-V Control Instructions

Control	*Conditional branches and jumps; PC-relative or through register*
`beq`, `bne`, `blt`, `bge`, `bltu`, `bgeu`	Branch GPR equal/not equal; less than; greater than or equal, signed and unsigned
`jal`, `jalr`	Jump and link: save PC+4, target is PC-relative (`JAL`) or a register (`JALR`); if specify `x0` as destination register, then acts as a simple jump
`ecall`	Make a request to the supporting execution environment, which is usually an OS
`ebreak`	Debuggers used to cause control to be transferred back to a debugging environment
`fence`, `fence.i`	Synchronize threads to guarantee ordering of memory accesses; synchronize instructions and data for stores to instruction memory

Readings

- H&P Computer Architecture Textbook chapter 1
- See end of chapter questions from H&P textbook
- RISC ISA details from Appendix A